Vehicle Sales Management System

Course: Advanced Database Management System Practical
Database: MySQL

Project Description

The Vehicle Sales Management System (VSMS) will be developed as a database project for storing and managing vehicle inventory, customer records, employee details, branch information, sales transactions and insurance policies. A self-created dataset will be used to complete database design, populated and tested using the concepts covered in the ADBMS practical.

Objectives

1. To enable the design of a structured dataset using proper data modelling techniques.
2. To develop practical skills in SQL for database creation, manipulation and querying.
3. To enhance analytical thinking through multi-phase database development.

Phase 1 - Data Modelling and Dataset Design

1. **Dataset**: A small VSMS dataset containing vehicles, customers, employees, branches, sales records and insurance policies. The same data will be used in all later phases

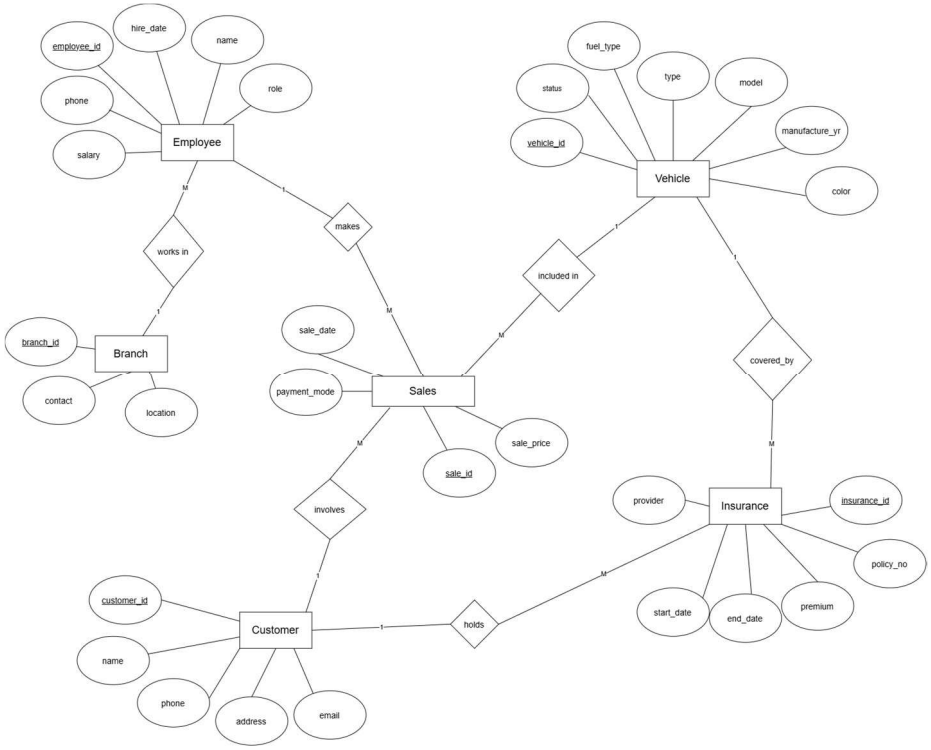
2. **Entity and attribute identification:** VEHICLE, CUSTOMER, SALES, EMPLOYEE, BRANCH and INSURANCE as the main entities, with attributes required to store their respective details.

3. **ER modelling:** VEHICLE-SALES, VEHICLE-INSURANCE, EMPLOYEE-SALES, CUSTOMER-INSURANCE, CUSTOMER-SALES and BRANCH-EMPLOYEE will be modelled mainly as 1:M relationships.

4. **Relational schema:** Convert the ER model into tables. Primary keys will identify records and foreign keys will connect related tables.

5. **Normalization:** customer, employee, vehicle and branch details will be stored separately so that repeated data is reduced and the schema remains in 3NF.

ER Diagram


employee_id
hire_date
name
phone
role
salary
Employee
works in
makes
branch_id
Branch
contact
location
sale_date
payment_mode
Sales
sale_price
sale_id
involves
customer_id
Customer
name
phone
address
email
holds
fuel_type
type
model
status
vehicle_id
manufacture_yr
Vehicle
color
included in
covered_by
provider
Insurance
insurance_id
policy_no
start_date
end_date
premium
1
M

Tables and Data Types

Table	Fields with Data Type	Relationship
VEHICLE	vehicle_id INT PK model VARCHAR(50) manufacturing_yr YEAR type VARCHAR(30) fuel_type VARCHAR(20) color VARCHAR(20) status VARCHAR(20)	VEHICLE 1:M SALES, VEHICLE 1:M INSURANCE
CUSTOMER	customer_id INT PK name VARCHAR(50) phone VARCHAR(15) email VARCHAR(50) address VARCHAR(100)	CUSTOMER 1:M SALES, CUSTOMER 1:M INSURANCE
EMPLOYEE	employee_id INT PK name VARCHAR(50) role VARCHAR(30) phone VARCHAR(15) salary DECIMAL(10,2) hire_date DATE branch_id INT FK	FK branch_id → BRANCH, EMPLOYEE 1:M SALES
BRANCH	branch_id INT PK location VARCHAR(100) contact VARCHAR(15) manager_id INT FK	BRANCH 1:M EMPLOYEE
SALES	sale_id INT PK vehicle_id INT FK customer_id INT FK employee_id INT FK sale_date DATE sale_price DECIMAL(10,2) payment_mode VARCHAR(20)	FK vehicle_id → VEHICLE, FK customer_id → CUSTOMER, FK employee_id → EMPLOYEE
INSURANCE	insurance_id INT PK vehicle_id INT FK customer_id INT FK provider VARCHAR(50) policy_no VARCHAR(30) start_date DATE end_date DATE premium DECIMAL(10,2)	FK vehicle_id → VEHICLE, FK customer_id → CUSTOMER

Relational Schema: BRANCH(branch_id) → EMPLOYEE(branch_id); EMPLOYEE(employee_id) → SALES(employee_id); VEHICLE(vehicle_id) → SALES(vehicle_id), INSURANCE(vehicle_id); CUSTOMER(customer_id) → SALES(customer_id), INSURANCE(customer_id).

Phase 2 - Database Creation and Data Preparation

1. **DDL**: Create VSMS_DB and all six tables using CREATE DATABASE and CREATE TABLE. PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints will be defined with the tables.
2. **DML**: Enter sample data using INSERT. UPDATE will be used when vehicle status or sale price changes, while DELETE will be tested only where foreign key relations are not affected.
3. **Data cleaning and constraints**: duplicate phone numbers and emails will be prevented, required IDs will not be NULL, and invalid negative sale price or premium values will be restricted.
4. **NULL handling**: payment_mode will handle cases where mode is not yet confirmed. IS NULL / IS NOT NULL will be used to find such records.
5. **Basic queries**: SELECT, WHERE and ORDER BY will be used to display available vehicles, completed sales, insurance records by provider and employees by branch.
6. **Aggregate functions**: COUNT() will calculate total sales per employee, AVG() will calculate average sale price and SUM() will calculate total revenue per branch.

Phase 3 - Data Querying and Analytical Operations

1. **Joins**: join CUSTOMER + SALES + VEHICLE for purchase history, EMPLOYEE + SALES for salesperson performance and VEHICLE + INSURANCE for coverage details.
2. **Nested queries and subqueries**: find customers who have made more than one purchase and vehicles whose sale price is above the overall average.
3. **View**: create Sales_Summary to display customer name, vehicle model, sale price, sale date and employee name through one reusable query.
4. **Query execution and optimization**: use EXPLAIN on the customer-sales-vehicle join. Indexes will be created on frequently searched columns such as SALES.vehicle_id and SALES.customer_id and the query plan will be checked again.
5. **Function**: create GetTotalSalesByEmployee(employee_id). It will receive an employee ID, count the matching records in SALES and return the total sales made by that employee.
6. **Procedure**: create AddSale(vehicle_id, customer_id, employee_id, sale_price, payment_mode). It will accept sale details and insert a new SALES record and update the vehicle status to Sold.Functional Modules

Function / Task	Tables Used	How I will do it
Register a customer	CUSTOMER	I will store customer name, contact and address using INSERT and retrieve records using SELECT with WHERE.
Add vehicle inventory	VEHICLE	I will insert vehicle details with status set to Available and update it to Sold after a sale is recorded.
Record a sale	SALES, EMPLOYEE, CUSTOMER, VEHICLE	I will select an available vehicle and customer, assign an employee and insert the sales record with payment details.
Assign Employee to Branch	EMPLOYEE, BRANCH	I will store branch_id as foreign key in the employee record to link each employees to their respective branch.
Record insurance policy	VEHICLE, INSURANCE, CUSTOMER	I will link the insurance record with vehicle_id and customer_id and store provider, policy number and premium details.
Generate sales report	VEHICLE, SALES, CUSTOMER, EMPLOYEE	I will use joins, aggregates and the Sales_Summary view to prepare revenue and performance summaries.

Phase 4 - Data Integrity and Transaction Handling

1. **Transaction handling**: when a sale is recorded, I will start one transaction, insert the SALES record and update VEHICLE.status to Sold. A SAVEPOINT will be used before the final update. COMMIT will be used if both operations succeed and ROLLBACK if an error occurs.

2. **Atomicity**: the sale insertion and vehicle status update will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database does not remain partly updated.

3. **Concurrency case**: before confirming a sale I will check that the selected vehicle status is still Available inside the transaction to prevent the same vehicle from being sold twice.

4. **Integrity constraints**: foreign keys will ensure that SALES uses an existing VEHICLE, CUSTOMER and EMPLOYEE, and INSURANCE uses an existing VEHICLE and CUSTOMER.

5. **Final validation**: check for missing references, confirm that sold vehicle records match status, and run basic queries, joins, subqueries, view, function, procedure and transaction cases to verify the database.

Outcome of the Project

1. A normalized relational database for vehicle sales records up to 3NF.
2. Practical implementation of DDL, DML, constraints, basic queries, aggregates, joins, subqueries, views, functions, procedures and transactions.
3. Vehicle, customer, employee, branch and insurance information can be stored and retrieved through related tables.
4. The project will give hands-on understanding of data modelling, SQL querying, integrity and transaction handling using one complete database case study.